

Spring Boot Internal Work

Q. How Spring Boot works internally?

```
6 @SpringBootApplication
7 // @EnableTransactionManagement
8 public class TransactionManagementApplication {
9
10     public static void main(String[] args) {
11         SpringApplication.run(TransactionManagementApplication.class, args);
12     }
13
14 }
15
```

run(...) Internal Flow

- Create application context
- Check Application Type
- Register the annotated class beans with the context
- Creates an instance of TomcatEmbeddedServletContainer and adds the context

Configure Multiple DataSource using Spring Boot

Github : <https://github.com/Java-Techie-jt/spring-boot-multiple-datasource>

- Add Oracle and MySQL JDBC driver dependencies in pom.xml

```

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>com.oracle.database.jdbc</groupId>
  <artifactId>ojdbc8</artifactId>
  <scope>runtime</scope>
</dependency>

```

1. Create a entity classes User and Book
2. Configure databases information in ***application.properties*** for 2 databases.
 1. url
 2. username
 3. password
 4. driver class name

```

1 spring.user.datasource.url=
2 spring.user.datasource.driver-class-name=
3 spring.user.datasource.username=
4 spring.user.datasource.password=
5
6 spring.book.datasource.url=
7 spring.book.datasource.driver-class-name=
8 spring.book.datasource.username=
9 spring.book.datasource.password=
10

```

3. Create a repositories for both User and Book.
4. Create a config class for User as below
5. Create the Datasource with help of the properties
6. Create a EntityManagerFactory with help of the datasource, properties
7. Create TrnasactionManager with help of the EntityManager
8. Do the same for Book and add any with **@Primary** annotation.

```

18
19 @Configuration
20 @EnableTransactionManagement
21 @EnableJpaRepositories(entityManagerFactoryRef = "entityManagerFactory", basePackages = {
22     "com.javatechie.multiple.ds.api.user.repository" })
23 public class UserDBConfig {
24
25     @Primary
26     @Bean(name = "datasource")
27     @ConfigurationProperties(prefix = "spring.user.datasource")
28     public DataSource dataSource() {
29         return DataSourceBuilder.create().build();
30     }
31
32     public LocalContainerEntityManagerFactoryBean entityManagerFactoryBean(EntityManagerFactoryBuilder builder,
33         @Qualifier("datasource") DataSource dataSource) {
34         Map<String, Object> properties = new HashMap<>();
35         properties.put("hibernate.hbm2ddl.auto", "update");
36         properties.put("hibernate.dialect", "org.hibernate.dialect.MYSQL5Dialect");
37         return builder.dataSource(dataSource).properties(properties)
38             .packages("com.javatechie.multiple.ds.api.model.user").persistenceUnit("User").build();
39     }
40 }
41

```

```

5
6     @Primary
7     @Bean(name = "transactionManager")
8     public PlatformTransactionManager transactionManager(
9         @Qualifier("entityManagerFactory") EntityManagerFactory entityManagerFactory) {
10         return new JpaTransactionManager(entityManagerFactory);
11     }
12
13 }

```

9. Add data with help of @PostConstruct annotation.

10. Write a API endpoints to read data from DB.

```

19
20 @SpringBootApplication
21 @RestController
22 public class SpringBootMultipleDsApplication {
23
24     @Autowired
25     private BookRepository bookRepository;
26
27     @Autowired
28     private UserRepository userRepository;
29
30     @PostConstruct
31     public void addData2DB() {
32         userRepository.saveAll(Stream.of(new User(744, "John"), new User(455, "Pitter")).collect(Collectors.toList()));
33         bookRepository.saveAll(
34             Stream.of(new Book(111, "Core Java"), new Book(222, "Spring Boot")).collect(Collectors.toList()));
35     }
36
37     @GetMapping("/getUsers")
38     public List<User> getUsers(){
39         return userRepository.findAll();
40     }
41
42     @GetMapping("/getBooks")
43     public List<Book> getBooks(){
44         return bookRepository.findAll();
45     }
46
47     public static void main(String[] args) {
48         SpringApplication.run(SpringBootMultipleDsApplication.class, args);
49     }
50 }

```